Pseudo-code

Outline

Introduction to pseudo-code

Variables and assignments

Basic operations

Basic constructs

Arrays

Functions

Sets

Style conventions

Introduction (1/3)

- In the previous chapter we have seen how the data of an algorithm, the numbers, are represented
- In this chapter we fix the language used to describe the algorithms themselves: the **pseudo-code**
- Pseudo-code is precise enough to describe every step of an algorithm without ambiguity, and simple enough to be read without knowing any programming language

Introduction (2/3)

The elements of pseudo-code, one at a time, from the simplest to the most structured:

- variables, declarations and assignments, and the basic operations on them
- the basic constructs that control the order of execution: conditional statements, loops, and the loop control statements **break** and **continue**
- arrays with one, two and more indices
- functions, which give a name to an algorithm, with its input and its output
- sets

Introduction (3/3)

Each construct is followed by examples,
executed step by step on small instances.

Executing an algorithm by hand, while keeping track
of the values of its variables, is the best way
to understand what it does.

Introduction to pseudo-code

Introduction: Introduction to pseudo-code

- In this section we define what an **algorithm** is and what **pseudo-code** is
- Pseudo-code is the language in which all the algorithms of the course are written
- We see its advantages, its key principles and the recommended practices for writing it

Algorithm and pseudo-code

Definition 1: algorithm

*An **algorithm** is a finite sequence of unambiguous instructions that, starting from some data given as **input**, produces a result given as **output** in a finite number of steps.*

Definition 2: pseudo-code

***Pseudo-code** is a high-level description of an algorithm that uses a mixture of natural language and programming language-like structures.*

What pseudo-code is for

- Pseudo-code is not meant to be executed by a computer, but rather to communicate the logic of an algorithm to humans
- Pseudo-code allows us to focus on the logic and structure of an algorithm without worrying about the syntax of a specific programming language
- **Convention:** all reserved words (keywords) of pseudo-code are written in bold, e.g., **if**, **while**, **for**, **return**, etc.

Advantages of pseudo-code

- Language-independent: it can be understood by programmers of different languages
- Easy to write and modify
- Focuses on the logic of the algorithm rather than on implementation details
- Facilitates communication between developers and non-technical stakeholders
- Useful for planning before actual coding

Key principles

- Use clear and descriptive variable names
- Keep statements simple and concise
- Use indentation to show structure and nesting
- Use standard constructs (**if-then-else**, **while**, **for**, etc.)
- Include comments when necessary to clarify complex operations

Recommended practices

- **Indentation:** use consistent indentation to show the block structure
- **Variables:** use descriptive names in italics or regular text
- **Comments:** add comments (in natural language) to explain complex steps
- **Input/Output:** clearly specify the input parameters and the output (return values)
- **Line numbers:** useful for referencing specific lines
- **Consistency:** maintain the same notation and style throughout

Summary

- An **algorithm** is a finite sequence of unambiguous instructions from an input to an output in a finite number of steps
- **Pseudo-code** describes an algorithm for humans, mixing natural language and programming language-like structures; keywords are in bold
- Good pseudo-code is clear, concise, consistently indented and commented, with input and output clearly specified

Variables and assignments

Introduction: Variables and assignments

- The data of an algorithm are stored in **variables**: named containers whose value can change during the execution
- A variable is first created with a **declaration** and then modified with **assignments**
- Declarations and assignments are the first statements of every algorithm

Variables

- Variables must be declared before use
- The declaration operator is ":="
- The declaration specifies the variable name and its initial value

```
1 variable := initial_value
```

Example: variable declaration

Seven variables declared with their initial values: integer, rational, $-\infty$ and logical (**true**/**false**) values are all allowed.

---

1 $n := 5$
2 $m := 10$
3 $x := -3/5$
4 $sum := 0$
5 $count := 1$
6 $max := -\infty$
7 $status :=$ true

Assignment

- The assignment operator is "$\leftarrow$"
- It assigns the value of an expression to a previously declared variable: first the expression on the right is evaluated, then its value replaces the old value of the variable

```
1 variable ← expression
```

Example: assignment

With the variables of the previous example:

1 $n \leftarrow 10$
2 $sum \leftarrow n + m$
3 $count \leftarrow count + 1$

- After the three assignments $n = 10$, $sum = 10 + 10 = 20$ and $count = 1 + 1 = 2$
- In the last assignment the same variable appears on both sides: the old value 1 is used to compute the new value 2

Example: exchanging the values of two variables

To exchange the values of two variables x and y we need a third, temporary variable *tmp*: the assignment $x \leftarrow y$ alone would overwrite the old value of x.

```
1 tmp := x
2 x ← y
3 y ← tmp
```

Step-by-step execution with $x = 3$ and $y = 8$:

statement	x	y	*tmp*
–	3	8	–
$tmp := x$	3	8	3
$x \leftarrow y$	8	8	3
$y \leftarrow tmp$	8	3	3

Summary

- A variable is **declared** with ":=", which gives its name and its initial value
- A variable is modified with an **assignment** "$\leftarrow$": the expression on the right is evaluated first, then its value replaces the old one
- To exchange the values of two variables a temporary variable is needed

Basic operations

Introduction: Basic operations

- Expressions combine variables and values with a small set of operations
- We list the mathematical, comparison and logical operations, and the input/output statements
- They appear on the right of assignments and in the conditions of the constructs that follow

Mathematical operations

- $+, -, \times, /$: basic arithmetic
- a mod b: modulo, i.e., the remainder of the integer division of a by b
- $\lfloor x \rfloor$: floor (greatest integer $\leq x$)
- $\lceil x \rceil$: ceiling (smallest integer $\geq x$)
- $|x|$: absolute value
- $\max(a, b)$, $\min(a, b)$: maximum and minimum

Comparison, logical and input/output operations

Comparison operators:

- $=, \neq$: equal, not equal
- $<, \leq, >, \geq$: less than, less than or equal to, greater than, greater than or equal to

Logical operators:

- **and**, **or**, **not**: logical operations

Input/Output:

- **read**(variable): read input from the user
- **print**(expression): output a value

Example: evaluating expressions

- $17 \bmod 5 = 2$, since $17 = 3 \cdot 5 + 2$; in particular, an integer i is a multiple of 3 if and only if $i \bmod 3 = 0$
- $\lfloor 7/2 \rfloor = \lfloor 3.5 \rfloor = 3$ and $\lceil 7/2 \rceil = \lceil 3.5 \rceil = 4$
- $|-3| = 3$, $\max(-3, 2) = 2$ and $\min(-3, 2) = -3$
- $(5 > 3)$ **and** $(2 > 4)$ is **false**, while $(5 > 3)$ **or** $(2 > 4)$ is **true**

Summary

- Mathematical operations: arithmetic, mod, floor $\lfloor \cdot \rfloor$, ceiling $\lceil \cdot \rceil$, absolute value, max and min
- Comparison operators ($=, \neq, <, \leq, >, \geq$) and logical operators (**and**, **or**, **not**) give **true** or **false**
- **read** and **print** exchange data with the user

Basic constructs

Introduction: Basic constructs

- The statements of pseudo-code are executed one after the other, from top to bottom
- Two kinds of constructs change this order:
  - **conditional statements**, which execute a block only when a condition holds
  - **loops**, which execute a block several times
- With them, and with **break** and **continue**, we can write every algorithm of the course

Conditional statements

We start with the conditional statements:
if-then and **if-then-else**.

If-then

- Executes a block of statements only if a condition is true

```
1 if condition then
2 |   statements
3 end
```

Example: if-then (1/2)

Checking whether a number x is positive:

```
1 positive := false
2 if x > 0 then
3 |   positive ← true
4 |   print(x is positive)
5 end
```

Example: if-then (2/2)

Step-by-step execution:

Instance 1: $x = 5$

x	$x > 0$?	*positive*
–	–	false
5	Yes	true

Output: `5 is positive`

Instance 2: $x = -3$

x	$x > 0$?	*positive*
–	–	false
-3	No	false

Output: nothing is printed.

If-then-else

- Executes one block if the condition is true, and another block if it is false

```
1 if condition then
2 |   statements when true
3 else
4 |   statements when false
5 end
```

Example: if-then-else (1/2)

Computing the absolute value of a number x:

```
1 result := 0
2 if x ≥ 0 then
3 |   result ← x
4 else
5 |   result ← −x
6 end
```

Example: if-then-else (2/2)

Step-by-step execution:

Instance 1: $x = 7$

x	$x \geq 0$?	*result*
–	–	0
7	Yes	7

Result: $result = 7$

Instance 2: $x = -4$

x	$x \geq 0$?	*result*
–	–	0
-4	No	4

Result: $result = 4$

Loops

We now move to the loops,
which execute a block several times:
the **while** loop and the **for** loop.

While loop

- Repeats a block of statements while a condition is true
- The condition is checked before each iteration
- If the condition is initially false, the loop body is never executed

```
1 while condition do
2 |   statements
3 end
```

Example: while loop (1/2)

Computing the sum of the numbers from 1 to n:

```
1 sum := 0
2 i := 1
3 while i ≤ n do
4 |   sum ← sum + i
5 |   i ← i + 1
6 end
```

Example: while loop (2/2)

Step-by-step execution. Instance: $n = 5$.

Each row shows the value of i when the condition is checked, the outcome of the check, and the values of *sum* and i at the end of the iteration.

i	$i \leq n$?	*sum*	i (after)
–	–	0	1
1	Yes	1	2
2	Yes	3	3
3	Yes	6	4
4	Yes	10	5
5	Yes	15	6
6	No	15	–

Result: $sum = 15$

When the number of iterations is not known

In the previous example the number of iterations is known in advance.

The while loop is necessary when it is not,
as in the next example.

Example: while loop (1/3)

Computing how many times a number x must be halved before it becomes smaller than or equal to a threshold $\varepsilon > 0$:

```
1 count := 0
2 while x > ε do
3 |   x ← x/2
4 |   count ← count + 1
5 end
```

Example: while loop (2/3)

Step-by-step execution. Instance 1: $x = 10$, $\varepsilon = 1$

x	*count*	$x > \varepsilon$?	x (after)	*count* (after)
10	0	Yes	5	1
5	1	Yes	2.5	2
2.5	2	Yes	1.25	3
1.25	3	Yes	0.625	4
0.625	4	No $\Rightarrow$ exit	–	4

Result: $count = 4$

Example: while loop (3/3)

Step-by-step execution. Instance 2: $x = 100$, $\varepsilon = 10$

x	*count*	$x > \varepsilon$?	x (after)	*count* (after)
100	0	Yes	50	1
50	1	Yes	25	2
25	2	Yes	12.5	3
12.5	3	Yes	6.25	4
6.25	4	No $\Rightarrow$ exit	–	4

Result: $count = 4$

For loop

- Repeats a block of statements for a specified number of times
- Typically used when the number of iterations is known in advance

```
1 for variable := start_value to end_value do
2 |   statements
3 end
```

The for loop as a while loop

The for loop is a compact way of writing a particular while loop: the loop variable is declared with the start value, the condition compares it with the end value, and it is increased by 1 at the end of each iteration.

The loop on the left is equivalent to the loop on the right:

```
1 for i := s to e do
2 |   statements
3 end
```

```
1 i := s
2 while i ≤ e do
3 |   statements
4 |   i ← i + 1
5 end
```

In particular, the body is executed $e - s + 1$ times if $s \leq e$, and it is never executed if $s > e$.

Example: for loop

Computing the sum of the numbers from 1 to n with a for loop (compare with the first example of the while loop):

```
1 sum := 0
2 for i := 1 to n do
3 |   sum ← sum + i
4 end
```

Instance: $n = 5$

i	$i \leq n$?	sum
–	–	0
1	Yes	1
2	Yes	3
3	Yes	6
4	Yes	10
5	Yes	15
6	No	15

Result: $sum = 15$

For loop with a step

The for loop can optionally use a **step** value,
to increase the loop variable by values other than 1.

Example: for loop with step

Computing the sum of the first n even numbers $(2, 4, \ldots, 2n)$:

```
1 sum := 0
2 for i := 2 to 2n step 2 do
3 |   sum ← sum + i
4 end
```

Instance: $n = 4$
(even numbers: $2, 4, 6, 8$)

i	$i \leq 2n$?	sum
–	–	0
2	Yes	2
4	Yes	6
6	Yes	12
8	Yes	20
10	No	20

Result: $sum = 20$

Loop control statements

Sometimes a loop must be interrupted before its natural end,
or a single iteration must be skipped.

Two statements serve these purposes:
break and **continue**.

Break statement (1/2)

- The **break** statement immediately exits the current loop
- Execution continues with the first statement after the loop
- Useful when a termination condition is met before the natural completion of the loop

Break statement (2/2)

```
1 while condition do
2 |   statements
3 |   if exit_condition then
4 |   |   statements
5 |   |   break
6 |   end
7 |   statements
8 end
```

```
1 for variable := start_value to
    end_value do
2 |   statements
3 |   if exit_condition then
4 |   |   statements
5 |   |   break
6 |   end
7 |   statements
8 end
```

Example: break statement (1/2)

Computing the sum of the numbers from 1 to n, stopping as soon as the sum exceeds a threshold t.

With a while loop:

```
1 sum := 0
2 i := 1
3 while i ≤ n do
4 |   sum ← sum + i
5 |   if sum > t then
6 |   |   break
7 |   end
8 |   i ← i + 1
9 end
```

With a for loop:

```
1 sum := 0
2 for i := 1 to n do
3 |   sum ← sum + i
4 |   if sum > t then
5 |   |   break
6 |   end
7 end
```

Example: break statement (2/2)

Step-by-step execution (both loops). Instance: $n = 6,\ t = 10$

i	sum	$sum > t$?
–	0	–
1	1	No
2	3	No
3	6	No
4	10	No
5	15	Yes $\Rightarrow$ **break**

Result: $sum = 15$; the iteration with $i = 6$ is never executed.

Continue statement (1/2)

- The **continue** statement skips the rest of the current iteration
- Execution proceeds immediately with the next iteration of the loop (in a while loop, with the check of the condition)
- Useful when certain iterations should be skipped based on a condition

Continue statement (2/2)

```
1 while condition do
2 |   statements
3 |   if skip_condition then
4 |   |   statements
5 |   |   continue
6 |   end
7 |   statements
8 end
```

```
1 for variable := start_value to
    end_value do
2 |   statements
3 |   if skip_condition then
4 |   |   statements
5 |   |   continue
6 |   end
7 |   statements
8 end
```

Example: continue statement (1/3)

Computing the sum of the numbers from 1 to n, skipping the multiples of 3.

With a while loop:

```
1  sum := 0
2  i := 1
3  while i ≤ n do
4  |   if i mod 3 = 0 then
5  |   |   i ← i + 1
6  |   |   continue
7  |   end
8  |   sum ← sum + i
9  |   i ← i + 1
10 end
```

With a for loop:

```
1 sum := 0
2 for i := 1 to n do
3 |   if i mod 3 = 0 then
4 |   |   continue
5 |   end
6 |   sum ← sum + i
7 end
```

Example: continue statement (2/3)

- In the while loop the variable i must be increased before **continue**
- Otherwise the condition $i \bmod 3 = 0$ would remain true forever and the loop would never end
- In the for loop the increase is automatic

Example: continue statement (3/3)

Step-by-step execution (both loops). Instance: $n = 7$

i	$i \bmod 3 = 0$?	sum
–	–	0
1	No	1
2	No	3
3	Yes	3
4	No	7
5	No	12
6	Yes	12
7	No	19

Result: $sum = 19 = 1 + 2 + 4 + 5 + 7$

Key points on the basic constructs

- **if-then** and **if-then-else** choose which block to execute according to a condition
- **while** repeats a block as long as a condition is true, checking it before each iteration; **for** repeats a block for each value of a loop variable from a start value to an end value
- **break** exits the loop; **continue** skips to the next iteration
- Executing a construct step by step on a small instance, with a table of the values of the variables, shows exactly what it does

Summary

- **Conditional statements**: if-then, if-then-else
- **Loops**: while (also when the number of iterations is not known in advance), for (with an optional step), and the equivalence of a for loop with a while loop
- **Loop control statements**: break and continue; in a while loop, the loop variable must be increased before **continue**

Arrays

Introduction: Arrays

- Many algorithms work on a collection of values of the same kind, for instance a list of n numbers
- Instead of declaring n different variables, we use an **array**
- We see arrays with one index, with two indices and with more indices: how to declare them, initialize them and access their elements

Declaring an array

- Arrays are indexed collections of variables and must be declared before use
- The declaration operator is "new array" followed by the index range

```
1 array_name := new array[start..end]
```

Example: array declaration. Creating an array of size n, indexed from 1 to n:

```
1 a := new array[1..n]
```

Accessing and initializing an array

- The notation $a[i]$ denotes the variable at index i
- Arrays are typically indexed starting from 0 or 1: in pseudo-code the indices start from 1, so an array of n elements is $a[1..n]$; in many programming languages, such as C++, the indices start from 0
- Each variable in the array must be initialized before access and use
- To initialize all the elements to the same value, use a **for** loop:

```
1 for i := start to end do
2 |   array_name[i] ← initial_value
3 end
```

Example: array assignment

Initializing and accessing the variables of an array $a[1..n]$ (with $n \geq 5$):

```
// Initialize all variables in the array to zero
1 for i := 1 to n do
2 |   a[i] ← 0
3 end

// Access and modify
4 x := a[5]
5 a[3] ← 42
```

With $n = 5$ the array is $[0, 0, 0, 0, 0]$ after the loop, $x = 0$, and the array is $[0, 0, 42, 0, 0]$ at the end.

Two-dimensional arrays

An array can also have **two indices**:
a row index and a column index.

Declaring a two-dimensional array

- Two-dimensional arrays are indexed collections of variables with two indices and must be declared before use
- The declaration operator is "new array" followed by the two index ranges
- Two-dimensional arrays are typically indexed starting from 0 or 1 (from 1 in pseudo-code)

```
1 array_name := new array[start_1..end_1][start_2..end_2]
```

Example: two-dimensional array declaration. Creating a two-dimensional array of size $m \times n$ (indexed from 1 to m and from 1 to n):

```
1 b := new array[1..m][1..n]
```

Accessing and initializing a two-dimensional array

- The notation $b[i][j]$ denotes the variable at row i and column j
- Each variable in the array must be initialized before access and use
- To initialize all the elements to the same value, use nested **for** loops:

```
1 for i := start_1 to end_1 do
2 |   for j := start_2 to end_2 do
3 |   |   array_name[i][j] ← initial_value
4 |   end
5 end
```

Example: two-dimensional array assignment

Initializing and accessing the variables of a two-dimensional array $b[1..m][1..n]$ (with $m \geq 2$ and $n \geq 3$):

```
// Initialize all variables in the two-dimensional
   array to zero
1 for i := 1 to m do
2 |   for j := 1 to n do
3 |   |   b[i][j] ← 0
4 |   end
5 end

// Access and modify
6 x := b[2][3]
7 b[1][1] ← 42
```

The order of the nested loops

- In the previous example the outer loop runs over the rows and, for each row i, the inner loop runs over all the columns
- The elements are visited in the order

$$b[1][1], b[1][2], \ldots, b[1][n], b[2][1], \ldots, b[m][n]$$

- for a total of $m \cdot n$ assignments

Multi-dimensional arrays

The same ideas extend to arrays with
any number of indices.

Declaring a multi-dimensional array

- Multi-dimensional arrays are indexed collections of variables with multiple indices and must be declared before use
- The declaration operator is "new array" followed by the index ranges for each dimension

```
1 array_name := new
  array[start_1..end_1][start_2..end_2]···[start_k..end_k]
```

Example: multi-dimensional array declaration. Creating a 3-dimensional array of size $p \times q \times r$ (indexed from 1 to p, from 1 to q, and from 1 to r):

```
1 t := new array[1..p][1..q][1..r]
```

Accessing and initializing a multi-dimensional array (1/2)

- The notation $t[i_1][i_2]\cdots[i_k]$ denotes the variable at indices $i_1, i_2, \ldots, i_k$
- Multi-dimensional arrays are typically indexed starting from 0 or 1 (from 1 in pseudo-code)
- Each variable in the array must be initialized before access and use

Accessing and initializing a multi-dimensional array (2/2)

To initialize all the elements to the same value, use nested **for** loops (one per dimension):

```
1 for i_1 := start_1 to end_1 do
2 |   for i_2 := start_2 to end_2 do
3 |   |   ⋮
4 |   |   for i_k := start_k to end_k do
5 |   |   |   array_name[i_1][i_2]···[i_k] ← initial_value
6 |   |   end
7 |   end
8 end
```

Example: multi-dimensional array assignment

Initializing and accessing a 3-dimensional array $t[1..p][1..q][1..r]$ (with $p \geq 2$, $q \geq 2$ and $r \geq 3$):

```
// Initialize all variables in the 3-dimensional
   array to zero
1 for i := 1 to p do
2 |   for j := 1 to q do
3 |   |   for k := 1 to r do
4 |   |   |   t[i][j][k] ← 0
5 |   |   end
6 |   end
7 end

// Access and modify
8 x := t[1][2][3]
9 t[2][1][3] ← 42
```

Summary

- An array is declared with **new array** and its index ranges: $a[1..n]$, $b[1..m][1..n]$, $t[1..p][1..q][1..r]$
- In pseudo-code the indices start from 1; in many programming languages, such as C++, from 0
- Every element must be initialized before use: one **for** loop per dimension, nested

Functions

Introduction: Functions

- The examples seen so far are fragments of pseudo-code that work on variables assumed to exist
- A **function** turns such a fragment into a complete algorithm, with a name, an input and an output
- From now on every algorithm of the course is written as a function: we see the syntax and five complete algorithms on arrays

Functions

- A **function** is a named, reusable block of code that performs a specific task
- A function takes **input** data through its **parameters** and produces **output** data
- The **return** statement terminates the function and specifies its results
- Functions are typically used to implement **algorithms** or to perform specific computations that can be reused in different parts of a program

Syntax of a function

Input : description of the input parameters
Output: description of the output / return value

```
1 function FunctionName(parameter_1, parameter_2, ..., parameter_k)
2 |   statements
3 |   return result
4 end
```

Algorithms as functions

- From now on every algorithm is written as a function, inside a red box
- The input states the **preconditions** on the data (for instance, that an array has at least one element)
- The function is guaranteed to work only when the preconditions hold

Example: maximum value in an array (1/4)

- A function returning the maximum value in an array
- The variable *max* starts from the first element and is updated whenever a larger element is found

Example: maximum value in an array (2/4)

Algorithm: FindMaximum

Input : An array $a[1..n]$ of $n \geq 1$ integer numbers
Output: The maximum value in the array a

```
1 function FindMaximum(a, n)
2 |   max := a[1]
3 |   for i := 2 to n do
4 |   |   if a[i] > max then
5 |   |   |   max ← a[i]
6 |   |   end
7 |   end
8 |   return max
9 end
```

Run in Colab: Python · C++ · Java

Example: maximum value in an array (3/4)

Step-by-step execution. Instance 1: $a = [9, 12, 3, 16, 19, 11, 12]$, $n = 7$

i	$a[i]$	$a[i] > max$?	max
–	–	–	9
2	12	Yes	12
3	3	No	12
4	16	Yes	16
5	19	Yes	19
6	11	No	19
7	12	No	19

Return: $max = 19$

Example: maximum value in an array (4/4)

Step-by-step execution. Instance 2: $a = [3, 7, 2, 9, 1]$, $n = 5$

i	$a[i]$	$a[i] > max$?	max
–	–	–	3
2	7	Yes	7
3	2	No	7
4	9	Yes	9
5	1	No	9

Return: $max = 9$

Example: counting the positive elements of an array (1/3)

A function counting the number of positive elements in an array:

Algorithm: CountPositive

Input : An array $a[1..n]$ of n integer numbers
Output : The number of positive elements in the array a

```
1 function CountPositive(a, n)
2 |   count := 0
3 |   for i := 1 to n do
4 |   |   if a[i] > 0 then
5 |   |   |   count ← count + 1
6 |   |   end
7 |   end
8 |   return count
9 end
```

Run in Colab: Python · C++ · Java

Example: counting the positive elements of an array (2/3)

Step-by-step execution. Instance 1: $a = [-2, 5, -1, 3, 0]$, $n = 5$

i	$a[i]$	$a[i] > 0$?	*count*
–	–	–	0
1	-2	No	0
2	5	Yes	1
3	-1	No	1
4	3	Yes	2
5	0	No	2

Return: $count = 2$

Example: counting the positive elements of an array (3/3)

Step-by-step execution. Instance 2: $a = [4, -3, 7, 2]$, $n = 4$

i	$a[i]$	$a[i] > 0$?	*count*
–	–	–	0
1	4	Yes	1
2	-3	No	1
3	7	Yes	2
4	2	Yes	3

Return: $count = 3$

Example: checking whether an array is sorted (1/3)

- A function checking whether an array is sorted in non-decreasing order
- The **return** statement inside the loop terminates the function as soon as two consecutive elements in the wrong order are found

Example: checking whether an array is sorted (2/3)

Algorithm: IsSorted

Input : An array $a[1..n]$ of $n \geq 1$ integer numbers
Output: **true** if the values in the array a are sorted in non-decreasing order and **false** otherwise

```
1 function IsSorted(a, n)
2 |   for i := 1 to n − 1 do
3 |   |   if a[i] > a[i + 1] then
4 |   |   |   return false
5 |   |   end
6 |   end
7 |   return true
8 end
```

Run in Colab: Python · C++ · Java

Example: checking whether an array is sorted (3/3)

Step-by-step execution.

Instance 1: $a = [2, 5, 7, 9]$, $n = 4$

i	$a[i]$	$a[i+1]$	$a[i] > a[i+1]$?
1	2	5	No
2	5	7	No
3	7	9	No

Return: true

Instance 2: $a = [3, 1, 4, 2]$, $n = 4$

i	$a[i]$	$a[i+1]$	$a[i] > a[i+1]$?
1	3	1	Yes

Return: false, after a single comparison.

Example: scalar product of two arrays (1/3)

A function computing the scalar product $\sum_{i=1}^{n} a[i] \cdot b[i]$ of two arrays:

Algorithm: ScalarProduct

Input : Two arrays $a[1..n]$ and $b[1..n]$ of size $n \geq 1$
Output: The scalar product of the two arrays

```
1 function ScalarProduct(a, b, n)
2 |   prod := a[1] · b[1]
3 |   for i := 2 to n do
4 |   |   prod ← prod + a[i] · b[i]
5 |   end
6 |   return prod
7 end
```

Run in Colab: Python · C++ · Java

Example: scalar product of two arrays (2/3)

Step-by-step execution. Instance 1: $a = [1, 2, 3]$, $b = [4, 5, 6]$, $n = 3$

i	$a[i]$	$b[i]$	$a[i] \cdot b[i]$	$prod$
–	–	–	–	$1 \cdot 4 = 4$
2	2	5	10	$4 + 10 = 14$
3	3	6	18	$14 + 18 = 32$

Return: $prod = 32$

Example: scalar product of two arrays (3/3)

Step-by-step execution. Instance 2: $a = [2, 0, -1]$, $b = [3, 4, 5]$, $n = 3$

i	$a[i]$	$b[i]$	$a[i] \cdot b[i]$	$prod$
–	–	–	–	$2 \cdot 3 = 6$
2	0	4	0	$6 + 0 = 6$
3	-1	5	-5	$6 + (-5) = 1$

Return: $prod = 1$

Example: reversing an array (1/4)

- A function reversing the elements of an array
- The element $a[i]$ is exchanged with its symmetric element $a[n - i + 1]$, with a temporary variable, as in the exchange of the values of two variables
- Only $\lfloor n/2 \rfloor$ exchanges are needed: with n odd, the middle element stays in place

Example: reversing an array (2/4)

Algorithm: ReverseArray

Input : An array $a[1..n]$ of n numbers
Output: The same array a with the elements in reversed order

```
1 function ReverseArray(a, n)
2 |   for i := 1 to ⌊n/2⌋ do
3 |   |   tmp := a[i]
4 |   |   a[i] ← a[n − i + 1]
5 |   |   a[n − i + 1] ← tmp
6 |   end
7 |   return a
8 end
```

Run in Colab: Python · C++ · Java

Example: reversing an array (3/4)

Step-by-step execution. Instance 1: $a = [1, 2, 3, 4, 5]$, $n = 5$, $\lfloor 5/2 \rfloor = 2$ iterations

i	exchange	array a (after the exchange)
–	–	$[1, 2, 3, 4, 5]$
1	$a[1] \leftrightarrow a[5]$	$[5, 2, 3, 4, 1]$
2	$a[2] \leftrightarrow a[4]$	$[5, 4, 3, 2, 1]$

Return: $a = [5, 4, 3, 2, 1]$

Example: reversing an array (4/4)

Step-by-step execution. Instance 2: $a = [10, 20, 30, 40]$, $n = 4$, $\lfloor 4/2 \rfloor = 2$ iterations

i	exchange	array a (after the exchange)
–	–	$[10, 20, 30, 40]$
1	$a[1] \leftrightarrow a[4]$	$[40, 20, 30, 10]$
2	$a[2] \leftrightarrow a[3]$	$[40, 30, 20, 10]$

Return: $a = [40, 30, 20, 10]$

Summary

- A **function** has a name, parameters, an input with its preconditions, an output, and a **return** statement that terminates it
- Five complete algorithms on arrays: FindMaximum, CountPositive, IsSorted, ScalarProduct, ReverseArray
- A **return** inside a loop terminates the function early, as in IsSorted

Sets

Introduction: Sets

- Arrays store values in positions
- When only the membership of the values matters, and not their position or their repetitions, we use a **set**
- We see how to declare a set, its main operators, and how to loop over its elements

Sets

- A **set** is an unordered collection of *distinct* elements
- Sets must be declared before use with the **new set** operator; no size needs to be specified; when created, a set is empty

```
1 set_name := new set
```

Operators on sets (1/2)

The main operators on sets are:

- **add**: insert an element into the set (ignored if already present)

  add x **to** set_name

- **remove**: delete an element from the set (ignored if not present)

  remove x **from** set_name

- **in**: membership test, which evaluates to **true** if x belongs to the set

  x **in** set_name

Operators on sets (2/2)

- **size**: cardinality (number of elements currently in the set)

size(set_name)

- **foreach**: loop over the elements of the set; by convention, the elements are visited in insertion order

```
1 foreach x in set_name do
  |  // process x
2 end
```

- It is possible to loop over the elements of a set in a specific order by specifying a sorting criterion:

```
1 foreach x in set_name sorted by criterion do
  |  // process x
2 end
```

Example: set declaration and usage (1/4)

```
   // Declare an empty set
1  s := new set

   // Populate the set with the even numbers from 1 to
      n
2  for i := 1 to n do
3  |   if i mod 2 = 0 then
4  |   |   add i to s
5  |   end
6  end

   // Membership test
7  if 4 in s then
8  |   print(4 is in s)
9  end
```

Example: set declaration and usage (2/4)

```
   // Remove an element
 9 remove 4 from s

   // Iterate and print all remaining elements
10 foreach x in s do
11 |   print(x)
12 end

   // Print the cardinality
13 print(size(s))
```

Example: set declaration and usage (3/4)

```
   // Print the elements in non-ascending order
13 foreach x in s sorted by value non-ascending do
14 |   print(x)
15 end

   // Print the elements in non-descending order
16 foreach x in s sorted by value non-descending do
17 |   print(x)
18 end
```

Example: set declaration and usage (4/4)

Step-by-step execution. Instance: $n = 8$

- Step 1: the for loop adds $2, 4, 6, 8$, so $s = \{2, 4, 6, 8\}$
- Step 2: 4 belongs to s: the output is `4 is in s`
- Step 3: after the removal, $s = \{2, 6, 8\}$
- Step 4: the first **foreach** prints $2, 6, 8$ (insertion order), then the cardinality 3 is printed
- Step 5: the last two loops print $8, 6, 2$ and $2, 6, 8$

Summary

- A **set** is an unordered collection of distinct elements, declared empty with **new set**
- Operators: **add**, **remove**, **in**, **size**
- **foreach** visits the elements in insertion order, or in the order given by **sorted by**

Style conventions

Introduction: Style conventions

- We conclude with a summary of the notation used in the course
- We compare it with other common choices, which you may find in books and in other courses

Different conventions

Different authors may use slightly different conventions:

- Declaration: $:=$ (used in this course) or $=$ (some authors use the same symbol for both declaration and assignment)
- Assignment: $\leftarrow$ (used in this course) or $=$
- Array indexing: $a[i]$ (used in this course) or a_i
- Two-dimensional array indexing: $b[i][j]$ (used in this course) or b_{ij}
- Comments: // comment, # comment, or (* comment *)

The important thing is to be consistent and clear!

Summary

- In this course: declaration $:=$, assignment $\leftarrow$, indexing $a[i]$ and $b[i][j]$
- Other authors may write $=$, a_i, b_{ij}, and other symbols for comments
- Whatever the notation, be **consistent and clear**

Summary: key points of the chapter

- Variables are declared with $:=$ and modified with $\leftarrow$
- Conditional statements and loops (**if**, **while**, **for**, with **break** and **continue**) control the order of execution
- Arrays $a[1..n]$, $b[1..m][1..n]$, ... are indexed from 1 in pseudo-code; sets store distinct elements without positions
- An algorithm is written as a **function** with its input (including the preconditions), its output and a **return** statement